

ui.html 全面详解

`ui.html` 是 NiceGUI 中用于直接嵌入原生 HTML 内容的核心组件，支持渲染任意 HTML 字符串、包含 CSS/JavaScript 脚本、集成第三方前端库等，继承自 `ui.element` 基类，具备动态更新、样式自定义、事件绑定等能力。该组件为开发者提供了最高灵活性，适用于原生 HTML 交互、第三方组件集成、复杂富文本展示等场景，是 NiceGUI 与原生 Web 技术桥接的关键工具。

一、核心定位与初始化参数

1. 核心定位

- 原生 HTML 渲染：直接解析并渲染 HTML 字符串，支持所有标准 HTML 标签（如 `<div>`、`<span>`、`<table>`、`<video>` 等）；
- 全栈集成能力：支持嵌入内联 CSS、JavaScript 脚本，可调用原生 DOM API 或集成第三方前端库（如 jQuery、Chart.js 等）；
- 灵活扩展：弥补内置组件的功能缺口，实现复杂交互逻辑（如自定义表单、动态 DOM 操作）；
- 继承基础能力：作为 `ui.element` 子类，支持 `classes`、`style`、`visible` 等通用属性及事件绑定方法。

2. 初始化参数

参数名	类型与说明	默认值	关键注意事项
content	待渲染的 HTML 字符串（支持单行 / 多行字符串、模板字符串）	-	核心必填参数，可包含 HTML 标签、内联 CSS（ <code><style></code> ）、JavaScript（ <code><script></code> ）
sanitize	HTML 净化函数（如 <code>Sanitizer().sanitize</code> ）或 <code>False</code> （禁用净化）	None	3.0+ 版本新增，用于过滤恶意 HTML/JS 防止 XSS 攻击，用户输入场景 强烈建议 启用
继承参数			继承自 <code>ui.element</code> 的 <code>classes</code> 、 <code>style</code> 、 <code>html_id</code> 、 <code>visible</code> 等属性

基础使用示例

```
from nicegui import ui

# 1. 简单 HTML 渲染（文本+标签）
ui.html('''
    <div style="color: #1e40af; font-size: 18px; font-weight: bold;">
        Hello, Native HTML!
    </div>
    <p>这是通过 ui.html 渲染的原生 HTML 内容。</p>
    <a href="https://nicegui.io" target="_blank">访问 NiceGUI 官网</a>
''')
```

```
# 2. 带净化的 HTML（用户输入场景）
from html_sanitizer import Sanitizer
user_input = '<script>alert("恶意脚本")</script><strong>安全的加粗文本</strong>'
ui.html(
    user_input,
    sanitize=Sanitizer().sanitize # 净化后仅保留 <strong> 标签，移除恶意脚本
)

ui.run()
```

二、核心功能与使用场景

1. 原生 HTML 元素渲染

支持所有标准 HTML 标签，包括文本、图片、表格、表单、媒体元素等，可直接复用现有 HTML 代码。

(1) 媒体元素（图片、视频、音频）

```
from nicegui import ui

ui.html('''
    <h3>媒体元素示例</h3>
    <!-- 图片 -->
    
    <!-- 视频 -->
    <video controls style="width: 400px; margin-top: 10px;">
        <source src="https://www.w3schools.com/html/mov_bbb.mp4" type="video/mp4">
        您的浏览器不支持视频播放
    </video>
    <!-- 音频 -->
    <audio controls style="width: 400px; margin-top: 10px;">
        <source src="https://www.w3schools.com/html/horse.ogg" type="audio/ogg">
        您的浏览器不支持音频播放
    ''')

ui.run()
```

(2) 复杂表格（合并单元格、样式定制）

```
from nicegui import ui

ui.html('''
    <h3>复杂表格示例</h3>
    <table border="1" cellpadding="8" cellspacing="0" style="border-collapse: collapse;
width: 100%;">
        <tr style="background-color: #f0f0f0;">
            <th colspan="2">表头 1（合并两列）</th>
            <th>表头 2</th>
        </tr>
        <tr>
            <td> </td>
            <td> </td>
        </tr>
    </table>
''')
```

```

        <td rowspan="2">表头 3（合并两行）</td>
        <td>单元格 1</td>
        <td>单元格 2</td>
    </tr>
    <tr>
        <td>单元格 3</td>
        <td>单元格 4</td>
    </tr>
</table>
'''

ui.run()

```

(3) 原生表单元素

```

from nicegui import ui

# 原生 HTML 表单 + NiceGUI 事件交互
ui.html('''
    <h3>原生表单示例</h3>
    <form id="native-form">
        <div style="margin-bottom: 10px;">
            <label for="name">姓名: </label>
            <input type="text" id="name" name="name" placeholder="请输入姓名" style="padding:
4px; width: 200px;">
        </div>
        <div style="margin-bottom: 10px;">
            <label for="age">年龄: </label>
            <input type="number" id="age" name="age" min="0" max="120" style="padding: 4px;
width: 200px;">
        </div>
        <button type="button" id="submit-btn" style="padding: 6px 12px; background-color:
#1e40af; color: white; border: none; border-radius: 4px;">
            提交
        </button>
    </form>
''')

# 绑定表单提交事件（通过 JavaScript 桥接）
ui.add_js('''
    // 给原生按钮绑定点击事件
    document.getElementById('submit-btn').addEventListener('click', () => {
        const name = document.getElementById('name').value;
        const age = document.getElementById('age').value;
        // 发送事件到 Python 侧
        emitEvent('form_submit', {name, age});
    });
''')

# Python 侧监听事件
ui.on('form_submit', lambda e: ui.notify(f'表单提交: 姓名={e.args["name"]}, 年龄=
{e.args["age"]}'))

```

```
ui.run()
```

2. 嵌入 CSS 样式（内联 / 内部）

支持通过 `<style>` 标签定义内部样式，或直接在元素上使用 `style` 属性设置内联样式，也可引用外部 CSS 文件。

```
from nicegui import ui

ui.html('''
    <style>
        /* 内部 CSS 样式 */
        .custom-card {
            background-color: #f8fafc;
            border: 1px solid #e2e8f0;
            border-radius: 8px;
            padding: 16px;
            margin-bottom: 16px;
            box-shadow: 0 1px 3px rgba(0,0,0,0.1);
        }
        .custom-title {
            color: #1e40af;
            font-size: 18px;
            margin-bottom: 8px;
        }
    </style>

    <div class="custom-card">
        <div class="custom-title">自定义样式卡片</div>
        <p>这是通过内部 CSS 定义样式的卡片内容。</p>
    </div>

    <!-- 引用外部 CSS 文件（需确保文件可访问） -->
    <link rel="stylesheet" href="/static/custom.css">
    <div class="external-style">外部 CSS 样式的内容</div>
''')

ui.run()
```

3. 嵌入 JavaScript 脚本（交互逻辑）

支持通过 `<script>` 标签嵌入 JavaScript 代码，实现原生 DOM 操作、事件绑定、异步请求等，可与 Python 侧通过 `emitEvent` 和 `on` 方法双向通信。

（1）原生 DOM 操作与 Python 交互

```
from nicegui import ui

ui.html('''
    <div id="counter" style="font-size: 24px; margin-bottom: 10px;">0</div>
    <button id="increment-btn" style="padding: 8px 16px;">+1</button>
''')
```

```

<button id="decrement-btn" style="padding: 8px 16px; margin-left: 10px;">-1</button>

<script>
  let count = 0;
  const counterEl = document.getElementById('counter');

  // 递增按钮事件
  document.getElementById('increment-btn').addEventListener('click', () => {
    count++;
    counterEl.textContent = count;
    // 发送计数到 Python 侧
    emitEvent('count_updated', count);
  });

  // 递减按钮事件
  document.getElementById('decrement-btn').addEventListener('click', () => {
    count--;
    counterEl.textContent = count;
    emitEvent('count_updated', count);
  });
</script>
''')

# Python 侧监听计数更新
ui.on('count_updated', lambda e: ui.notify(f'当前计数: {e.args}'))

ui.run()

```

(2) 集成第三方 JavaScript 库

通过 `<script>` 标签引入第三方库（如 Chart.js、jQuery），快速实现复杂功能（如图表渲染、DOM 简化操作）。

```

from nicegui import ui

# 集成 Chart.js 绘制图表
ui.html('''
  <!-- 引入 Chart.js 库 -->
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>

  <div style="width: 80%; margin: 0 auto;">
    <canvas id="myChart"></canvas>
  </div>

  <script>
    // 初始化图表
    const ctx = document.getElementById('myChart').getContext('2d');
    new Chart(ctx, {
      type: 'bar',
      data: {
        labels: ['周一', '周二', '周三', '周四', '周五', '周六', '周日'],
        datasets: [{
          label: '访问量',
          data: [1200, 1900, 1500, 2100, 2500, 2200, 1800],

```

```

        backgroundColor: 'rgba(30, 64, 175, 0.6)',
        borderColor: 'rgba(30, 64, 175, 1)',
        borderWidth: 1
    }]
},
options: {
    responsive: true,
    scales: {
        y: { beginAtZero: true }
    }
}
});
</script>
'''

ui.run()

```

4. 动态更新 HTML 内容

支持通过 `content` 属性或 `set_content` 方法动态修改 HTML 内容，实现交互性更新（如根据用户操作切换内容）。

```

from nicegui import ui

# 创建 HTML 组件并保存引用
html_component = ui.html('''
    <div style="text-align: center; padding: 20px;">
        <h3>初始内容</h3>
        <p>点击按钮切换内容</p>
    </div>
''')

# 切换到内容 1
def show_content1():
    html_component.set_content('''
        <div style="text-align: center; padding: 20px; background-color: #f0f9ff;">
            <h3>内容 1</h3>
            <p>这是动态加载的内容 1</p>
            
        </div>
    ''')

# 切换到内容 2
def show_content2():
    html_component.set_content('''
        <div style="text-align: center; padding: 20px; background-color: #fef7fb;">
            <h3>内容 2</h3>
            <p>这是动态加载的内容 2</p>
            
        </div>
    ''')

```

```
# 按钮控制切换
with ui.row(classes="justify-center mt-4"):
    ui.button('显示内容 1', on_click=show_content1)
    ui.button('显示内容 2', on_click=show_content2)

ui.run()
```

5. 安全净化 (防 XSS 攻击)

3.0+ 版本新增 `sanitize` 参数, 用于过滤 HTML 中的恶意脚本、危险标签 (如 `<script>`、`<iframe>`), 保护应用安全。需安装 `html-sanitizer` 包 (`pip install html-sanitizer`)。

```
from nicegui import ui
from html_sanitizer import Sanitizer

# 配置净化规则 (自定义允许的标签和属性)
custom_sanitizer = Sanitizer({
    'tags': {
        'a': {'href', 'target'},
        'strong': set(),
        'em': set(),
        'img': {'src', 'alt'},
    },
    'attributes': {'a': {'href', 'target'}, 'img': {'src', 'alt'}},
    'protocols': {'a': {'href': ['http', 'https', 'mailto']}},
})

# 恶意用户输入 (包含脚本和危险标签)
malicious_input = '''
    <script>alert('XSS 攻击')</script>
    <strong>安全文本</strong>
    <a href="javascript:恶意代码">危险链接</a>
    <a href="https://nicegui.io" target="_blank">安全链接</a>
    
'''

# 净化后渲染 (移除 <script> 和 javascript: 链接)
ui.html(malicious_input, sanitize=custom_sanitizer.sanitize)

ui.run()
```

净化规则说明:

- `tags`: 允许的 HTML 标签集合;
- `attributes`: 允许的标签属性集合;
- `protocols`: 允许的 URL 协议 (如仅允许 `http/https`);
- 未指定的标签 / 属性 / 协议会被自动移除, 避免恶意代码执行。

三、组件属性

`ui.html` 继承自 `ui.element`，除基础属性外，新增 `content` 和 `sanitize` 绑定属性，支持动态同步内容和净化规则：

属性名	类型	说明	适用场景
content	BindableProperty	HTML 内容（可绑定到其他对象属性，支持双向同步）	动态内容更新、状态同步
sanitize	Callable False	HTML 净化函数 (3.0+)，设为 False 禁用净化	安全渲染用户输入内容
classes	Classes[Self]	组件的 CSS 类 (Tailwind/Quasar 类)，用于整体样式调整	组件布局、背景、边距配置
style	Style[Self]	内联 CSS 样式，用于精细化样式调整	组件整体尺寸、对齐方式
html_id	str	HTML DOM 元素 ID (NiceGUI 2.16.0+ 新增)	原生 JS 交互、CSS 选择器定位
visible	BindableProperty	组件可见性 (布尔值，支持绑定)	条件显示 / 隐藏 HTML 内容

属性使用示例（内容绑定）

```
from nicegui import ui
from html_sanitizer import Sanitizer

# 定义状态类
class AppState:
    def __init__(self):
        self.html_content = '<strong>初始 HTML 内容</strong>'

state = AppState()

# HTML 内容双向绑定 + 净化
html_component = ui.html('') \
    .bind_content(state, 'html_content') \
    .sanitize(Sanitizer().sanitize)

# 输入框修改内容，同步到 HTML 组件
ui.textarea(
    label="编辑 HTML 内容",
    value=state.html_content,
    rows=3,
    on_change=lambda e: setattr(state, 'html_content', e.value)
).classes('w-full mt-4')

ui.run()
```


四、核心方法

除继承 `ui.element` 的所有方法（如 `move`、`delete`、`tooltip` 等）外，`ui.html` 新增专属方法用于内容操作和净化配置：

方法名	参数与说明	功能描述
<code>set_content(content)</code>	<code>content</code> : 新的 HTML 字符串	手动设置组件内容（动态更新）
<code>set_sanitize(sanitize)</code>	<code>sanitize</code> : 净化函数或 <code>False</code>	动态设置净化规则（3.0+ 支持）
<code>bind_content</code>	<code>target_object</code> : 目标对象； <code>target_name</code> : 属性名； <code>forward/backward</code> : 转换函数	内容双向绑定到目标对象属性（组件 ↔ 目标对象）
<code>bind_content_from</code>	<code>target_object</code> : 目标对象； <code>target_name</code> : 属性名； <code>backward</code> : 转换函数	内容单向绑定（目标对象 → 组件）

方法使用示例（动态切换净化规则）

```
from nicegui import ui
from html_sanitizer import Sanitizer

# 配置两种净化规则
strict_sanitizer = Sanitizer({'tags': {'strong', 'em'}}) # 仅允许粗体和斜体
relaxed_sanitizer = Sanitizer({'tags': {'strong', 'em', 'a', 'img'}}) # 允许更多标签

# 初始 HTML 内容
html_content = '''
    <strong>粗体文本</strong>
    <em>斜体文本</em>
    <a href="https://nicegui.io">链接</a>
    
'''

html_component = ui.html(html_content, sanitize=strict_sanitizer.sanitize)

# 切换净化规则
def switch_sanitizer(relaxed: bool):
    if relaxed:
        html_component.set_sanitize(relaxed_sanitizer.sanitize)
        ui.notify('已切换到宽松净化规则（允许链接和图片）')
    else:
        html_component.set_sanitize(strict_sanitizer.sanitize)
        ui.notify('已切换到严格净化规则（仅允许粗体和斜体）')
    html_component.set_content(html_content) # 重新渲染内容

# 开关控制净化规则
ui.switch(
    label='宽松净化规则',
    on_change=lambda e: switch_sanitizer(e.value)
```

```
    ).classes('mt-4')

    ui.run()
```

五、版本兼容性与依赖说明

功能 / 属性	最低版本要求	依赖说明
基础 HTML 渲染	-	无额外依赖，内置支持原生 HTML 解析
sanitize 参数	3.0.0	需安装 <code>html-sanitizer</code> (<code>pip install html-sanitizer</code>)，用于 HTML 净化
html_id 属性	2.16.0	新增 HTML DOM 元素 ID 配置能力
双向绑定 content	3.0.0	支持 <code>bind_content</code> 方法，低版本需手动通过 <code>set_content</code> 更新

依赖安装命令

```
# 基础依赖（默认已安装）
pip install nicegui

# 安全净化依赖（3.0+ 版本，用户输入场景必备）
pip install html-sanitizer

# 确保 NiceGUI 版本兼容（建议 ≥3.0.0 以支持完整功能）
pip install nicegui>=3.0.0
```

六、常见问题与注意事项

1. 安全风险（重点关注）

- XSS 攻击防护：用户输入场景**必须启用** `sanitize` 参数，禁止直接渲染未净化的用户输入（如评论、表单提交内容）；
- 第三方脚本风险：引入外部 JavaScript 库或 CSS 文件时，确保来源可信，避免恶意代码注入；
- `sanitize=False` 禁用净化时，仅适用于**完全信任的 HTML 内容**（如开发者手动编写的代码），禁止用于用户输入。

2. 样式冲突

- 优先级：HTML 内联 `style` > 内部 `<style>` > NiceGUI 全局样式 > Tailwind/Quasar 类；
- 解决方法：使用唯一 CSS 类名（如前缀 `custom-`），避免与 NiceGUI 内置类冲突；通过 `!important` 强制覆盖样式（谨慎使用）。

3. 事件绑定与通信

- 原生 JS 与 Python 通信：通过 `emitEvent` (JS 侧) 和 `ui.on` (Python 侧) 实现，确保事件名称唯一；
- DOM 加载顺序：如果 JS 脚本操作 DOM 元素，需确保元素已渲染（可通过 `DOMContentLoaded` 事件或延迟执行）。

4. 动态更新问题

- 动态修改内容后，需调用 `set_content` 方法刷新组件；如果内容包含 JS 脚本，新脚本会重新执行；
- 复杂 DOM 结构动态更新时，建议先清空现有内容（如 `set_content('')`），再加载新内容，避免残留 DOM 元素。

5. 路径问题

- 引用本地资源（图片、CSS、JS 文件）时，需将文件放在 `static` 目录下（NiceGUI 自动识别该目录为静态资源目录），路径写为 `/static/文件名`；
- 示例：本地图片 `static/logo.png`，HTML 中引用为 ``。

七、高级应用场景示例

1. 集成第三方富文本编辑器 (TinyMCE)

```
from nicegui import ui

# 集成 TinyMCE 富文本编辑器
ui.html('''
    <!-- 引入 TinyMCE 库 -->
    <script src="https://cdn.tiny.cloud/1/no-api-key/tinymce/7/tinymce.min.js"
referrerpolicy="origin"></script>

    <textarea id="editor"></textarea>

    <script>
        // 初始化 TinyMCE
        tinymce.init({
            selector: '#editor',
            height: 300,
            plugins: 'advlist autolink lists link image charmap print preview anchor',
            toolbar: 'undo redo | bold italic | alignleft aligncenter alignright | bullist
numlist outdent indent | link image',
            setup: function(editor) {
                // 编辑器内容变化时，同步到 Python 侧
                editor.on('change', function() {
                    emitEvent('editor_content', editor.getContent());
                });
            }
        });
    </script>
''')
```

```
# 显示编辑器内容预览
preview = ui.html('<div style="margin-top: 20px; padding: 10px; border: 1px solid #e2e8f0;">
预览区: </div>')

# Python 侧监听编辑器内容变化, 更新预览
ui.on('editor_content', lambda e: preview.set_content(f'''
    <div style="margin-top: 20px; padding: 10px; border: 1px solid #e2e8f0;">
        预览区: <br>{e.args}
    </div>
'''))

ui.run()
```

2. 原生 HTML 模态框 (无需 NiceGUI 内置组件)

```
from nicegui import ui

ui.html('''
<style>
    /* 模态框样式 */
    .modal {
        display: none;
        position: fixed;
        top: 0;
        left: 0;
        width: 100%;
        height: 100%;
        background-color: rgba(0,0,0,0.5);
        z-index: 1000;
        align-items: center;
        justify-content: center;
    }
    .modal-content {
        background-color: white;
        padding: 20px;
        border-radius: 8px;
        width: 400px;
        max-width: 90%;
    }
    .close-btn {
        margin-top: 15px;
        padding: 8px 16px;
        background-color: #ef4444;
        color: white;
        border: none;
        border-radius: 4px;
        cursor: pointer;
    }
</style>

<!-- 触发按钮 -->
```

```

<button onclick="openModal()" style="padding: 10px 20px; background-color: #1e40af;
color: white; border: none; border-radius: 4px;">
    打开原生模态框
</button>

<!-- 模态框 -->
<div id="myModal" class="modal">
    <div class="modal-content">
        <h3>原生 HTML 模态框</h3>
        <p>这是通过 HTML/CSS/JS 实现的模态框，无需依赖 NiceGUI 内置组件。</p>
        <button class="close-btn" onclick="closeModal()">关闭</button>
    </div>
</div>

<script>
    const modal = document.getElementById('myModal');

    function openModal() {
        modal.style.display = 'flex';
    }

    function closeModal() {
        modal.style.display = 'none';
        // 关闭后通知 Python 侧
        emitEvent('modal_closed', '模态框已关闭');
    }

    // 点击模态框外部关闭
    window.addEventListener('click', (e) => {
        if (e.target === modal) closeModal();
    });
</script>
'''

# Python 侧监听模态框关闭事件
ui.on('modal_closed', lambda e: ui.notify(e.args))

ui.run()

```

3. 动态加载远程 HTML 内容

```

from nicegui import ui
import requests

# 加载远程 HTML 内容（示例：加载 NiceGUI 官网文档片段）
def load_remote_html():
    try:
        # 发起请求获取远程 HTML
        response = requests.get('https://nicegui.io/documentation/html', timeout=5)
        response.raise_for_status()
        # 提取文档核心内容（简化示例，实际需解析 DOM）
        html_content = f'''

```

```
        <div style="padding: 20px; background-color: #f8fafc; border-radius: 8px;">
            <h3>远程加载的 HTML 内容</h3>
            <p>以下是 NiceGUI ui.html 文档的部分内容: </p>
            <hr>
            {response.text[:2000]}... <!-- 截取前 2000 字符 -->
        </div>
    '''

    html_component.set_content(html_content)
except Exception as e:
    html_component.set_content(f'<div style="color: red;">加载失败: {str(e)}</div>')

# 初始内容
html_component = ui.html('<div style="text-align: center; padding: 50px;">点击按钮加载远程 HTML</div>')

# 加载按钮
ui.button('加载远程 HTML', on_click=load_remote_html).classes('mt-4')

ui.run()
```

总结

`ui.html` 是 NiceGUI 中灵活性最高的组件之一，核心优势在于：

1. **原生兼容性**：直接渲染任意 HTML 内容，无缝集成现有 Web 技术栈；
2. **全栈交互**：支持 HTML/CSS/JS 完整生态，可与 Python 侧双向通信；
3. **功能扩展**：弥补内置组件缺口，集成第三方库（如图表、富文本编辑器）；
4. **安全可控**：3.0+ 版本提供 `sanitize` 参数，有效防范 XSS 攻击。

使用时需重点关注安全问题，尤其是处理用户输入时必须启用 HTML 净化；同时注意样式冲突和 DOM 加载顺序，确保交互逻辑正常运行。无论是原生 Web 组件集成、复杂富文本展示还是第三方库对接，`ui.html` 都能提供高效、灵活的解决方案，是 NiceGUI 开发中不可或缺的高级工具。